

Building ML web apps with Gradio

Gradio를 활용한 ML 웹 앱 제작

03 [Project1] 이미지 분류 웹 앱 만들기

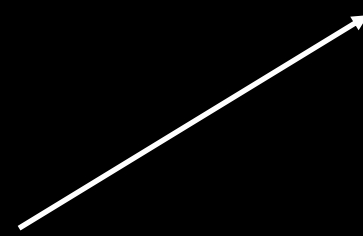
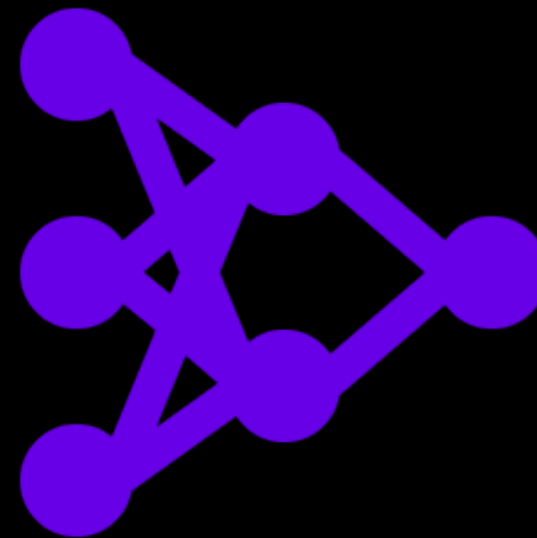
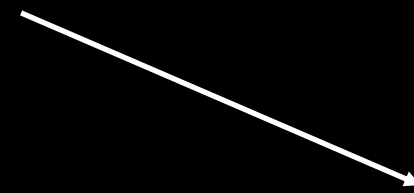
목차 03 [Project1] 이미지 분류 웹 앱 만들기

1. 이미지 분류 모델
2. 데이터셋 소개
3. 이미지 업로드 및 분류

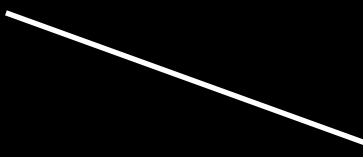
1. 이미지 분류 모델



- 이미지 분류 모델은 주어진 이미지를 여러 클래스 중 하나로 분류하는 머신러닝 모델
- 특정 작업이나 환경에 따라 적합한 모델을 선택하는 것이 중요
- 사전 훈련된 모델을 사용하여 전이 학습(Transfer Learning)을 적용하는 것이 일반적이며, 이는 적은 데이터로도 효과적인 이미지 분류 모델을 만들 수 있게 함



Dog



Cat



이미지 분류 사례

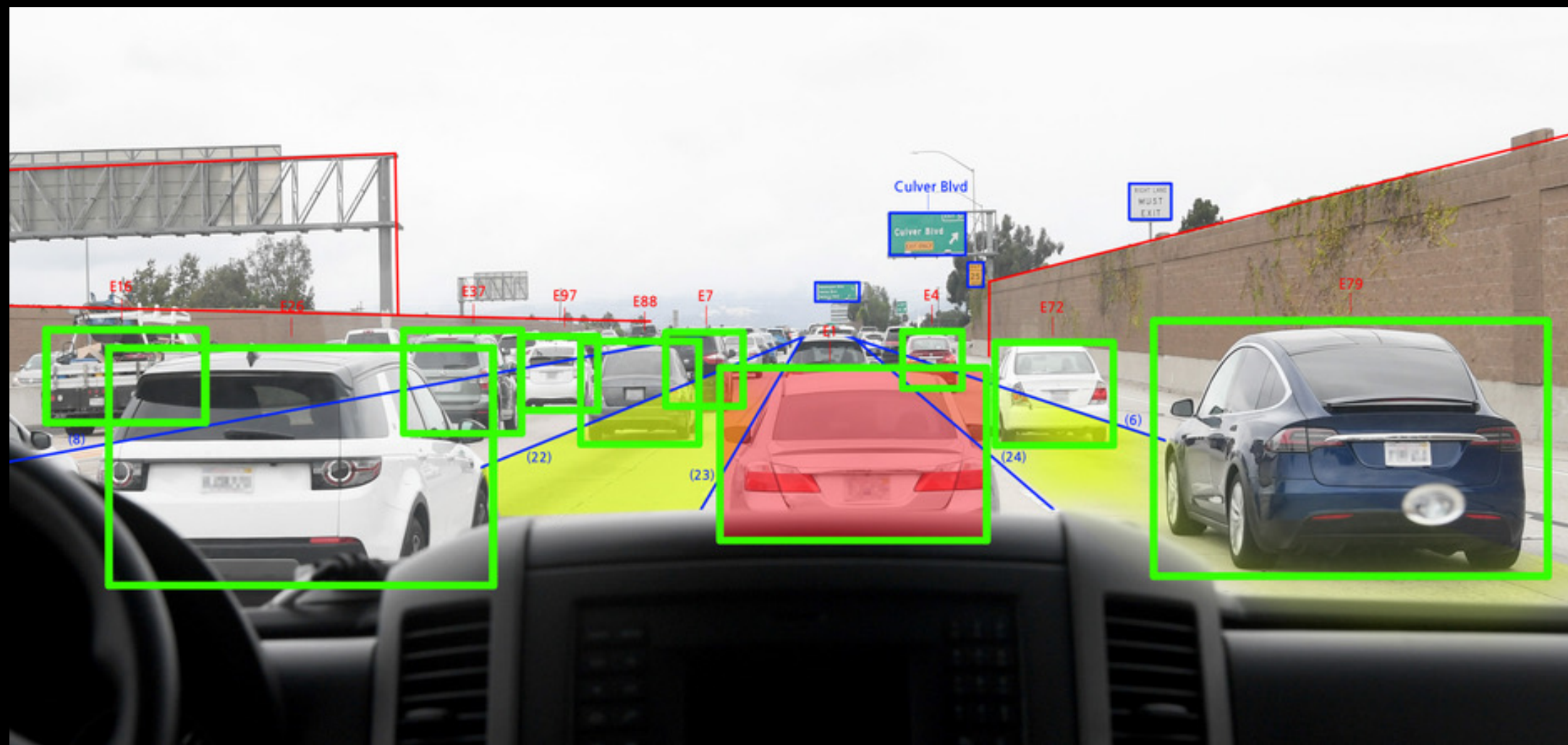
Gradio를 활용한
ML 웹 앱 제작

03 [Project1] 이미지
분류 웹 앱 만들기

1. 이미지 분류 모델



의료 진단



자율 주행



고양이와 개 분류하기

Gradio를 활용한
ML 웹 앱 제작

03 [Project1] 이미지
분류 웹 앱 만들기

1. 이미지 분류 모델

inp



↑

📍

📄

Clear

Submit

output

dog

dog 60%

cat 40%

Flag

2. 데이터셋 소개



Cat and Dog

Gradio를 활용한
ML 웹 앱 제작

03 [Project1] 이미지
분류 웹 앱 만들기

2. 데이터셋 소개

- 1,500장 RGB 이미지
- 이진 분류 (1 = dog, 0 = cat)



3. 이미지 업로드 및 분류

Gradio의 주요 구성요소로, 모델을 중심으로 웹 기반 GUI/데모를 만들도록 지원

매개 변수로는 1) 함수 2) 입력 컴포넌트 3) 출력 컴포넌트 등이 있음

```
import gradio as gr

def image_classifier(inp):
    return {'cat': 0.3, 'dog': 0.7}
```

```
demo = gr.Interface(fn=image_classifier, inputs="image", outputs="label")
demo.launch()
```



Interface의 4가지 유형

Gradio를 활용한
ML 웹 앱 제작

03 [Project1] 이미지
분류 웹 앱 만들기

3. 이미지 업로드 및 분류

```
gr.Interface(fn, inputs, outputs)
```

Input(s)



Output(s)

Standard Interface

```
gr.Interface(fn, None, outputs)
```

Output(s)

Output-Only Interface (e.g. GAN)

```
gr.Interface(fn, inputs, None)
```

Input(s)

Input-Only Interface (e.g. Data Collector)

```
gr.Interface(fn, inputs, inputs)
```

Input(s) / Output(s) ↻

Unified Interface (e.g. Text Generator)



Interface의 필수 매개변수

Gradio를 활용한
ML 웹 앱 제작

03 [Project1] 이미지
분류 웹 앱 만들기

3. 이미지 업로드 및 분류

```
import gradio as gr
```

```
def image_classifier(inp):
```

```
    return {'cat': 0.3, 'dog': 0.7}
```

1) 함수

2) 입력 컴포넌트

3) 출력 컴포넌트

```
demo = gr.Interface(fn=image_classifier, inputs="image", outputs="label")  
demo.launch()
```

input_img


이미지를 끌어 놓으세요

Clear

Submit

output



Flag



Interface의 필수 매개변수

Gradio를 활용한
ML 웹 앱 제작

03 [Project1] 이미지
분류 웹 앱 만들기

3. 이미지 업로드 및 분류

```
import gradio as gr
```

```
def image_classifier(inp):
```

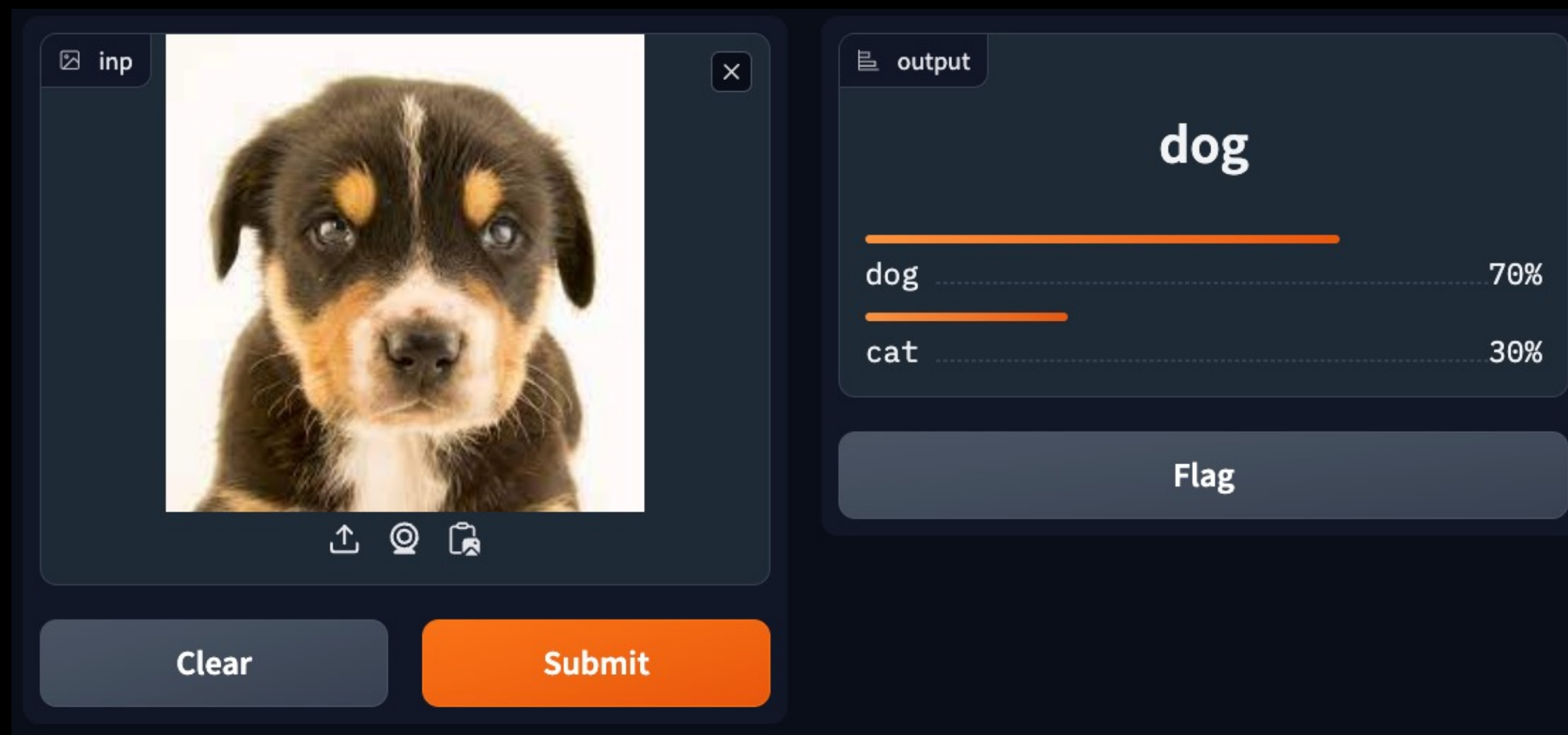
```
    return {'cat': 0.3, 'dog': 0.7}
```

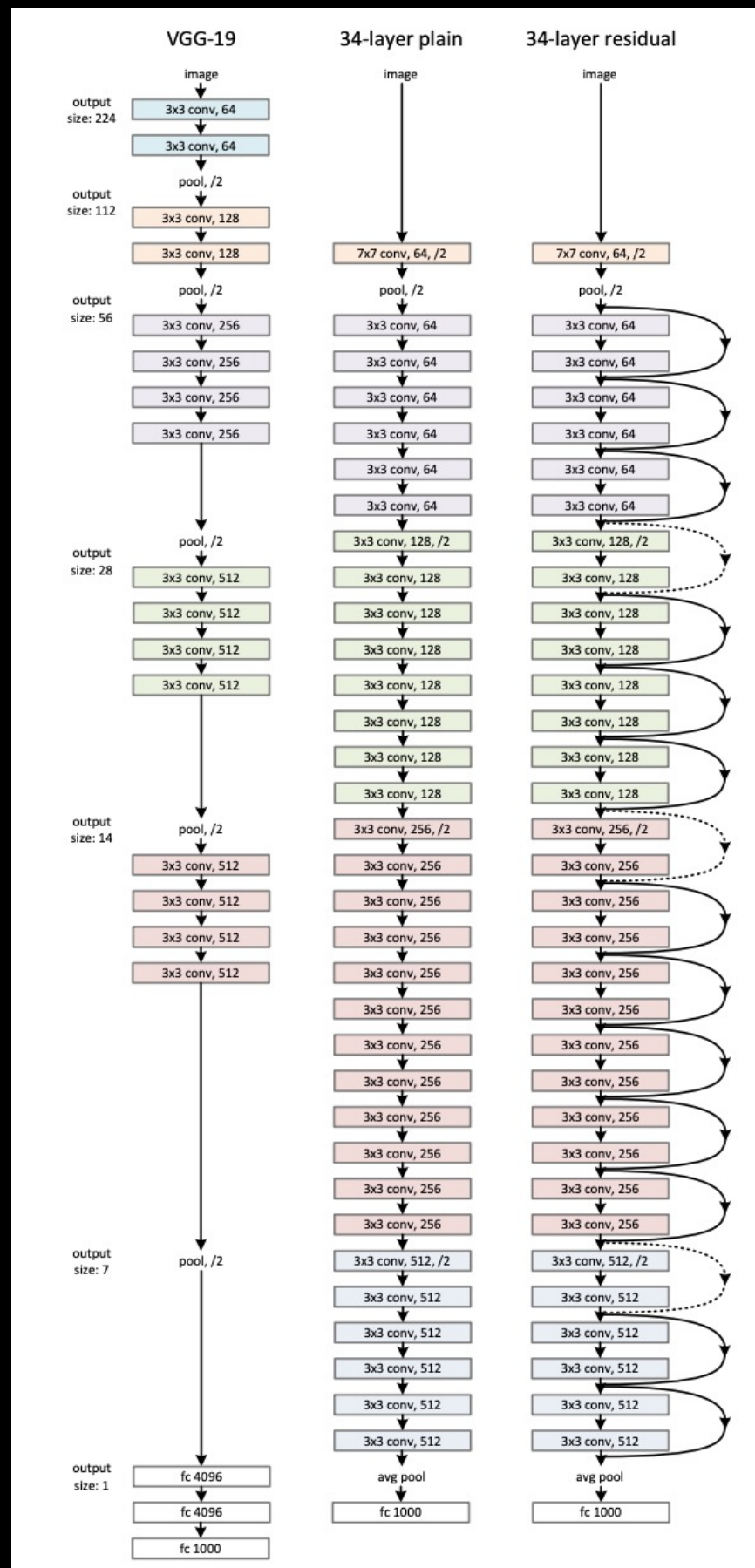
1) 함수

2) 입력 컴포넌트

3) 출력 컴포넌트

```
demo = gr.Interface(fn=image_classifier, inputs="image", outputs="label")  
demo.launch()
```





- 이미지 분류 모델로 ResNet50을 선정
- Torchvision에서 불러올 수 있는 사전학습 모델 중에서 정확도와 파라미터 수의 트레이드 오프를 고려

사전 학습(Pretraining)은 기계 학습의 한 단계

대규모의 데이터셋에서 모델을 미리 학습시키는 과정

이 단계에서 모델은 특정 작업에 대한 세부적인 정보를 학습하기 전에

입력 데이터의 일반적인 특징이나 구조 등을 파악





```
model = torch.hub.load('pytorch/vision:v0.6.0',  
                        'resnet50', pretrained=True).eval()
```

Torchvision에서 ImageNet 데이터로 사전 학습한 ResNet50 모델을 다운로드
pretrained=True → 사전 학습한 모델



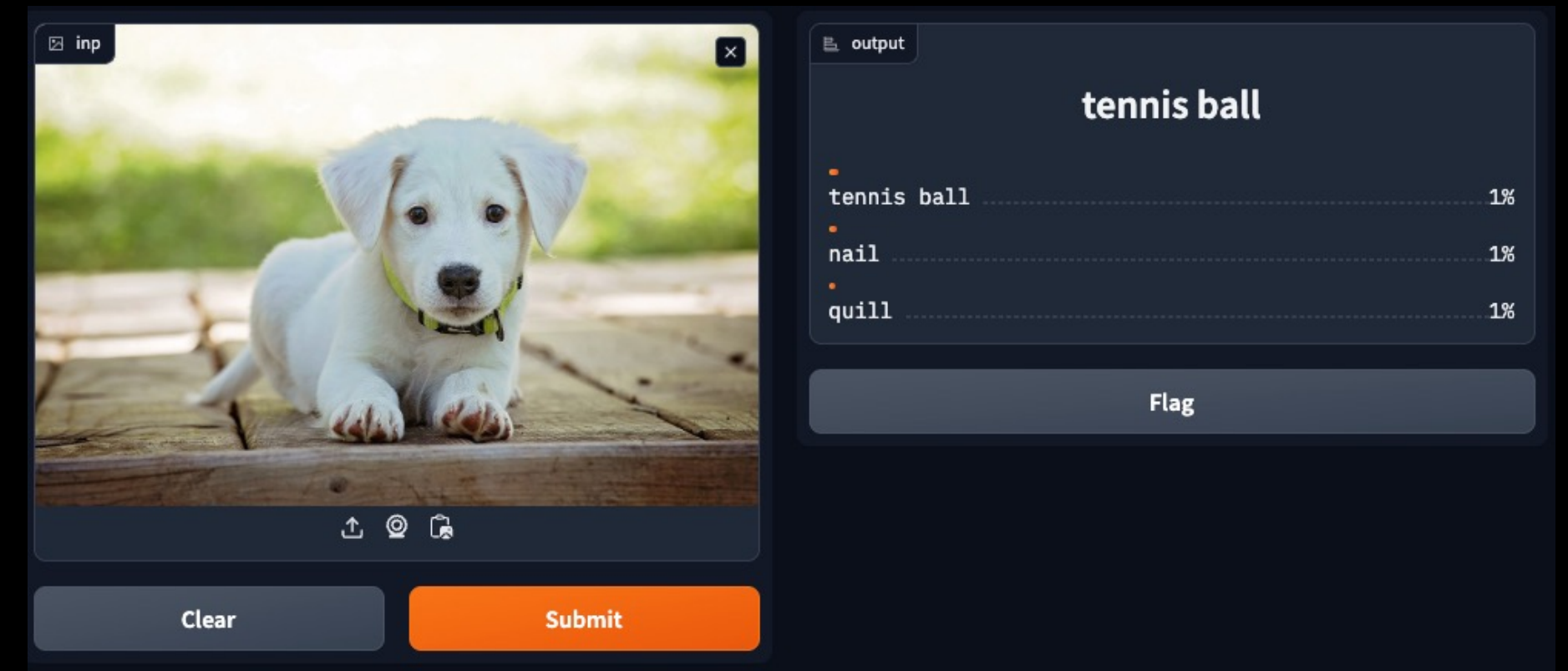
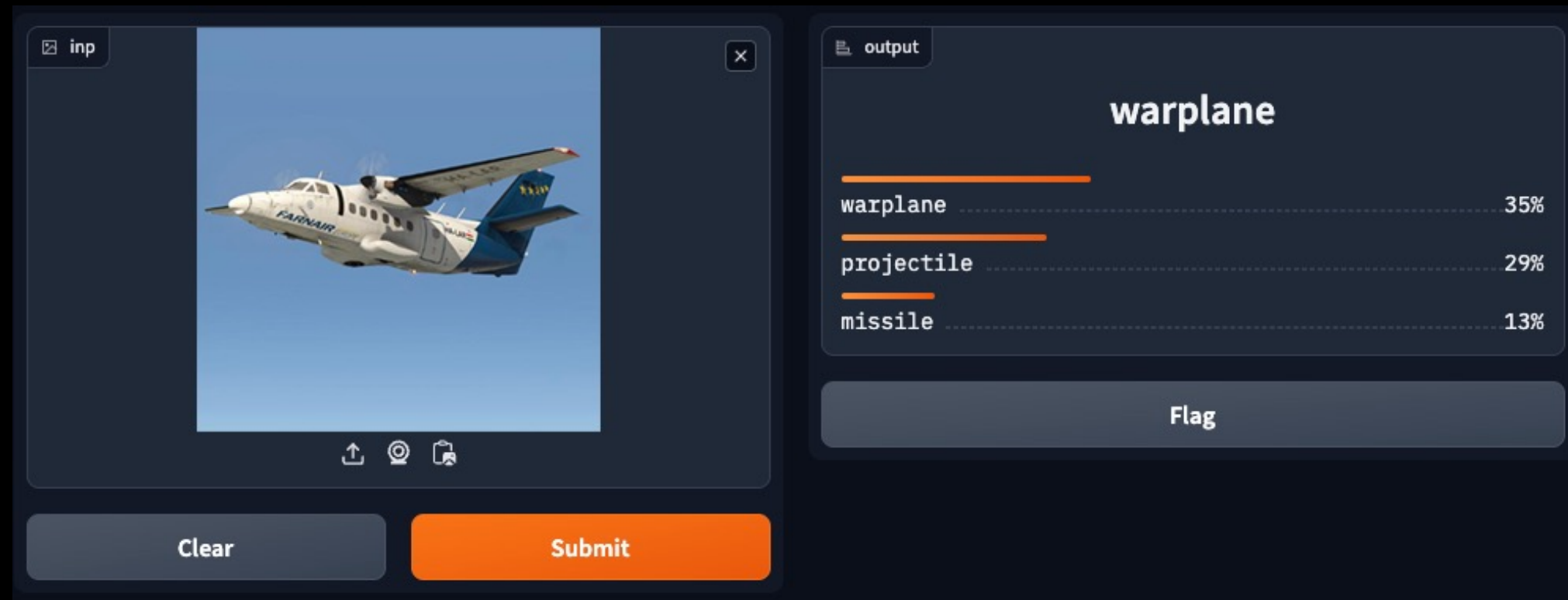
이미지 업로드 및 분류 결과 확인

Gradio를 활용한
ML 웹 앱 제작

03 [Project1] 이미지
분류 웹 앱 만들기

3. 이미지 업로드 및 분류

• 실습



전투기는 잘 맞히지만, 강아지는 잘 맞히지 못함
사전학습 되지 않은 이미지도 맞히게 할 수 있을까?

Fine-tuning

: 모델의 파라미터를 미세하게 조정하는 것으로,
사전 학습한 모델에 추가로 학습하여 파라미터를 업데이트

➡ Dog and Cat 데이터셋으로 Fine-tuning을 하게 되면 성능이 향상